Recreate ragcap from zero

Written for: an engineer who has never seen this project and wants to rebuild it, run every scenario, and check the result against the recorded fingerprints. Time: about 45 minutes, plus 40 minutes if you also run the answer-level evaluation. Date of this guide: 2026-10-19.

ragcap is a reproduction of Kumar's "RAG Capstone" design (LangGraph router, document RAG with grade and rewrite, SQL agent over uploaded tables, general agent, Postgres + pgvector, FastAPI, Gradio) built on local models, with two things his PDF did not have: a frozen evaluation set with quality metrics, and a version-aware filter that keeps superseded documents out of retrieval. Everything below was run on a 2019 Intel MacBook Pro (12 cores, 17 GB RAM, macOS 15.1.1).

0. What you need before starting

Need	Why	Check
Docker Desktop (any recent version; 20.10.7 was used)	Postgres + pgvector run in a container because this Mac has no Homebrew	`open -a Docker` then `docker info` prints a server version
Ollama with `llama3.2:3b` and `nomic-embed-text`	the LLM and the embedding model, both local, no API key	`ollama list` shows both; `ollama pull <name>` if not
Python 3.12	the venv pins; 3.13+ has no wheels for some packages on Intel macOS	`python3.12 --version` or any 3.12 interpreter path
10 GB free disk, lid open	models (8.5 GB) plus the Postgres image; the Mac must not sleep during runs	`caffeinate -i` is used by every long script

Not needed: Homebrew, an API key, a GPU, Docling (it cannot run on this hardware, see section 6).

1. Get the code

The code is public at https://github.com/shreenathacc22/ragcap: `git clone https://github.com/shreenathacc22/ragcap && cd ragcap`. The first commit is the golden-set freeze and there is one commit per step after it (see fingerprints). Never copy `.env`, `.venv`, `uploads/` or `traces.jsonl` from another machine: the rebuild makes new ones.

Path	What it is
`ragcap/config.py`	every setting: Postgres DSN from `.env`, model names, `TOP_K=6`, `VERSION_AWARE`, grader switches
`ragcap/db.py`, `ragcap/llm.py`	one Postgres connection helper (pgvector registered); OpenAI-compatible client to Ollama; `embed()`
`ragcap/ingest.py`	file to Markdown to chunks to vectors; front-matter to metadata; CSV/Excel to SQL tables
`ragcap/graph.py`	the LangGraph: router, retrieve, grade, rewrite, generate, sql_agent, generate_sql, general; `ask()`
`ragcap/tracing.py`	one JSONL line per question in `traces.jsonl`
`schema.sql`, `step1_check.py`	tables `documents`, `chunks` (vector(768), HNSW), `uploaded_tables`; the step 1 proof
`golden/`	`golden.jsonl` (27 cases, tuned on), `golden_heldout.jsonl` (25 unseen), `amendments.jsonl`, generators
`eval.py`, `b1_eval.py`, `b1_check.py`, `summarize_series.py`	answer-level evaluation; retrieval-only OFF/ON ablation; data checks; series aggregation
`run_series.sh`, `run_answer_ablation.sh`, `recreate.sh`	long runs under `caffeinate`; the one-command rebuild

Path	What it is
`api.py`, `ui.py`, `try.py`	FastAPI, Gradio, shell probe
`data/docs_versioned/` (16 files), `data/tables/broker_inventory.csv`	the corpus: 8 Kafka runbooks x old/current, and one table
`results/`	every evaluation result that was recorded, by name and timestamp

2. The build, step by step

Each step has Do, Expect, Verify. `recreate.sh` runs steps 2.1 to 2.6 unattended and checks the fingerprints; read this section once so you know what it is doing.

2.1 Database: Postgres 16 + pgvector in Docker

Do:

```
cd ragcap                       # the clone
PW=$(openssl rand -hex 12)
printf 'PGHOST=localhost\nPGPORT=5433\nPGUSER=ragcap\nPGPASSWORD=%s\nPGDATABASE=ragcap\n' "$PW" > .env; chmod 600 .env
docker run -d --name ragcap-pg -e POSTGRES_USER=ragcap -e POSTGRES_PASSWORD="$PW" -e POSTGRES_DB=ragcap \
  -p 127.0.0.1:5433:5432 -v ragcap_pgdata:/var/lib/postgresql/data --restart unless-stopped pgvector/pgvector:pg16
sleep 8; docker exec ragcap-pg psql -U ragcap -d ragcap -c "CREATE EXTENSION IF NOT EXISTS vector;"
```

Expect: `CREATE EXTENSION`. Verify: `docker exec ragcap-pg psql -U ragcap -d ragcap -tAc "select extversion from pg_extension where extname='vector'"` prints `0.8.7` (any 0.8.x is fine; 0.8 is needed for `hnsw.iterative_scan`).

Why port 5433 on 127.0.0.1: no clash with a local Postgres, and the database is unreachable from the network. The password is fixed on first creation of the volume; if you lose `.env`, remove the volume and start again (runbook section 4).

2.2 Python environment and schema

Do:

```
python3.12 -m venv .venv            # or: <path-to-any-3.12>/python -m venv .venv
.venv/bin/pip install -r requirements.txt
.venv/bin/python step1_check.py
```

Expect, last lines:

```
tables: ['chunks', 'documents', 'uploaded_tables']
embedding dim: 768
nearest: Kafka consumer lag alert threshold 0.383
STEP 1 OK
```

Verify: `STEP 1 OK`. The check applies `schema.sql`, embeds two sentences with nomic-embed-text, stores them, and confirms a cosine search returns the Kafka one for a lag question. If the dimension assertion fails, the embedding model is not the one in `config.py`.

2.3 Ingest the versioned corpus and the table

Do:

```
.venv/bin/python -m ragcap.ingest data/docs_versioned
.venv/bin/python -m ragcap.ingest data/tables
```

Expect: 16 lines like `{'file': 'consumer_lag_v1.md', 'doc': 'Kafka Consumer Lag Runbook', 'version': 1, 'superseded': True, 'chunks': 1, ...}` and one line `{'table': 'broker_inventory', 'rows': 6, 'columns': [...]}`.

Verify:

```
docker exec ragcap-pg psql -U ragcap -d ragcap -tAc "select (select count(*) from documents)||' docs, '|| \
  (select count(*) from chunks)||' chunks, '||(select count(*) from documents where superseded)||' superseded, '|| \
  (select count(*) from uploaded_tables)||' tables'"
```

prints `16 docs, 16 chunks, 8 superseded, 1 tables`.

What happens inside: each `.md` starts with a front-matter block (`doc`, `version`, `effective_date`, `superseded`, `status`). It is parsed into the `documents` row and stripped before embedding, so the word SUPERSEDED never reaches the vector (if it did, the old and new versions would no longer look alike and the version experiment would be meaningless). The document title is prepended to each chunk before embedding so an orphan paragraph still says what it is about. Each runbook is short enough to be one chunk.

2.4 Data checks and negative tests

Do: `.venv/bin/python b1_check.py --no-trap --negative` then `.venv/bin/python b1_check.py`

Expect: `PASS` both times. The second run also prints the similarity trap: pair similarity 0.962 to 0.990 for the eight old/current pairs, and `old version ranked first for 1/8 plain questions`. The negative run must print `reported` for all three injected faults (old marked current, version as text, banner in text). If a negative test prints `NOT REPORTED`, the checker is broken and nothing downstream can be trusted.

2.5 Smoke the graph: one question per route

Do: `.venv/bin/python try.py "Thanks!" "At what disk utilisation should a broker page?" "How many brokers are in rack-b?"`

Expect (times are for this CPU; a 3B model):

```
Q: Thanks!                            route=general    path: router > general
   A: You're welcome! ...
Q: At what disk utilisation should ...  route=documents  path: router > retrieve > grade > generate
   A: ... 75% ...
Q: How many brokers are in rack-b?      route=database   path: router > sql_agent > generate_sql
   sql: SELECT COUNT(*) FROM broker_inventory WHERE rack = 'rack-b'     A: There are 2 ...
```

Note: `VERSION_AWARE` defaults to on (1) since the ablation was recorded (step 20g). Run the same question with `VERSION_AWARE=0` to see the trap: the answer may come back with 85%, the retired threshold (scenario V1 in section 3).

2.6 The retrieval ablation: filter OFF then ON

Do: `caffeinate -i .venv/bin/python b1_eval.py --latency 3` (about one minute; no LLM, so it is deterministic)

Expect, exactly:

```
OFF: superseded 99/192 (51.6%) | trap 2/12 (stale from old chunk 10, any stale string 10) | control 20/20
ON : superseded 0/192 (0.0%)  | trap 12/12 (stale from old chunk 0, any stale string 6) | control 20/20
latency: {'rounds': 3, 'calls_each': 96, 'off_median_ms': ~56, 'on_median_ms': ~57}
```

Verify: `results/ablation_summary.md` has `0/192 (0.0%)` in the ON column. Latency medians will differ by machine; OFF and ON must be within a few ms of each other.

2.7 Answer-level evaluation (LLM in the loop)

Do: `caffeinate -i .venv/bin/python eval.py` (27 cases, 15 to 25 minutes) or `caffeinate -i .venv/bin/python eval.py --golden golden/golden_heldout.jsonl --tag heldout`. For the mean over repeated runs: `./run_series.sh <label> llama3.2:3b` then `.venv/bin/python summarize_series.py <label>`. For the version filter at answer level: `./run_answer_ablation.sh`.

Expect (llama3.2:3b, recorded ranges; a different model or a different day will not reproduce these exactly, which is why they are ranges): original set router 100%, answers 85 to 93%, retrieval hit 100%; held-out set router 96%, answers 80 to 84%, retrieval hit 100%; the same five held-out ids fail every time (h-doc-04/05/08/15, h-db-01).

2.8 API and UI

Do: `.venv/bin/uvicorn api:app --port 8000 &` then `curl -s localhost:8000/health`; `.venv/bin/python ui.py` opens Gradio at http://127.0.0.1:7860.

Expect: `/health` returns `{"documents":16,"chunks":16,"tables":["broker_inventory"]}`. `POST /chat` streams one `node` event per graph node then an `answer` event (route, citations, SQL, total ms). `POST /upload` ingests a file. Both are unauthenticated: localhost only.

2.9 Front end, LangSmith (optional)

Do: `.venv/bin/uvicorn api:app --port 8000 &` then open http://127.0.0.1:8000/ in a browser, ask `Which brokers are above 85% disk usage?`. Expect: three pills appear one by one (`router`, `sql_agent`, `generate_sql`, each with its ms), then the answer, `route: database`, and the SQL. The page is `web/index.html`, served by FastAPI; it reads `POST /chat` as server-sent events. LangSmith: add `LANGSMITH_TRACING=true`, `LANGSMITH_API_KEY`, `LANGSMITH_PROJECT=ragcap` to `.env` and restart; without them nothing changes.

2.10 P&ID branch (drawings)

Do: `ollama pull qwen2.5vl:3b` (3.2 GB, once), then `.venv/bin/python pid/make_pid.py` (three synthetic drawings plus `pid/ground_truth.json` and `pid/golden_pid.jsonl`, deterministic), then `.venv/bin/python pid_eval.py --ingest` (about 5 minutes to describe the drawings on this CPU, then 12 questions through the graph).

Expect: `results/pid_eval_summary.md` with a tag-extraction table per drawing and the QA scores (see SOP step 24 for the recorded numbers and what they mean). A described drawing is one more document in the index; `try.py "Which valve isolates the suction of pump P-101?"` retrieves `PID-001` and answers `HV-101`.

3. Scenario catalogue

Run any scenario with `.venv/bin/python try.py "<question>"` (prefix `VERSION_AWARE=1` where stated). "Path" is the node sequence printed by `try.py`; "Shows" is what the scenario demonstrates. Expected answers are for llama3.2:3b; the facts must appear, the wording will vary.

Id	Question	Expected route and path	Expected answer contains	Shows
G1	`Thanks!`	general: router > general	a short acknowledgement, no facts	the general agent never states facts; 2 LLM calls, ~4 s
G2	`What is the largest message size our brokers accept?`	documents: router > retrieve > grade > rewrite > retrieve > grade > generate	`not in the knowledge base`	the grade/rewrite loop: nothing relevant, one rewrite, then an explicit refusal instead of a guess (Kumar's "document question with query rewrite" flow)
D1	`At what consumer lag should operations raise an alert?`	documents: router > retrieve > grade > generate	`5,000 messages` (not 10,000)	the normal document flow with a citation [1]
D2	`What port does BROKER-EAST-02 use for TLS traffic?`	documents	`9093` (not 9092)	an exact-string lookup; identical in both versions, so it is a control case for the version filter
D3	`What replication factor and what default retention apply to a new standard production topic?`	documents	`3` and `72 hours`	multi-hop: two facts from two documents in one answer (fails about half the time on the 3B model: the per-passage grader keeps one topic)

Id	Question	Expected route and path	Expected answer contains	Shows
S1	`Which brokers are above 85% disk usage?`	database: router > sql_agent > generate_sql	brokers 3 and 5 (87.2%, 91.8%)	the SQL agent: a SELECT with a WHERE from the question, run read-only with a 5 s timeout, then worded from the rows
S2	`Which broker hosts the most topics, and how many?`	database	broker 5, 210	ORDER BY ... LIMIT 1 for a single extreme; run 3 answered this with a wrong GROUP BY, which is why the SQL prompt has the rule
S3	`What is the total disk capacity across all brokers in GB?`	database	`16,000` or `16000`	SUM; the answer step must copy numbers exactly (run 1 wrote "16 GB")
S4	`delete all brokers` (or any non-SELECT)	database; sql_agent rejects	`could not compute ... only a single read-only SELECT is allowed`	the validator and the READ ONLY transaction: nothing but one SELECT can run
V1 OFF	`VERSION_AWARE=0` + `At what disk utilisation should a broker page?`	documents	often `85%` (the retired threshold)	the trap: the old runbook (v1, 85%) is 0.99-similar to the current one (v2, 75%) and is retrieved 51.6% of the time
V1 ON	`VERSION_AWARE=1` + the same question	documents	`75%`	the fix: `WHERE d.superseded = false` before ranking; the old version cannot be retrieved at all
V2	`VERSION_AWARE=1` + `Which team owns escalation for a SEV-1 Kafka incident?`	documents	`Streaming Platform team` (never `Platform Reliability`)	a version conflict with no number in it: only metadata can separate the versions
R1	`What is the highest disk usage percentage and which broker has it?`	expected database; observed documents (known failure h-db-01)	wrong: `75%` from the runbook instead of 91.8% from the table	router ambiguity between a rule in a document and a measurement in a table; fails 3/3 runs
R2	`How many minutes must consumer lag stay high before the alert fires?`	documents; observed `not in the knowledge base` (known failure h-doc-05)	wrong: should be `5 minutes`	the 3B grader rejects a passage phrased differently from the question ("sustained for 5 minutes"); fails 3/3 runs; a stronger grader is the fix (SOP step 19 for why it was not possible here)
W1	(browser) `Which brokers are above 85% disk usage?` at http://127.0.0.1:8000/	database; pills router > sql_agent > generate_sql	brokers 3 and 5	the streaming contract: one server-sent event per node, then the answer; any front end consumes the same endpoint

Id	Question	Expected route and path	Expected answer contains	Shows
M1	`Which valve isolates the suction of pump P-101?` (after 2.10)	documents; retrieves `PID-001`	`HV-101` (not HV-102)	a drawing answered through its vision description; the chunk's meta carries the image path
M2	`Which instrument controls the flow into tank T-102, and through which valve?`	documents	`FIC-101`, `FCV-101`	the hardest tag family for a 3B vision model (see SOP step 24: the control-valve tags were the ones misread)
U1	`Which cloud provider hosts the Kafka cluster?`	documents, with rewrite	`not in the knowledge base`	unanswerable: refused, not guessed (2/2 in every run)

Where to see the evidence for each: `traces.jsonl` (one line per question: route, per-node ms, tokens, answer), `results/version_filter_off.json` and `_on.json` (per case: which documents were retrieved, marked `OLD`), `results/eval_*.json` (per case: route, answer, pass/fail).

4. Fingerprints a correct rebuild must match

What	Value
`golden/golden.jsonl` sha1	`68ec09bdfcf99d62999ccfbe708f6f8ac3eaa4d3` (27 cases; frozen in commit db9af51 before any code)
`golden/golden_heldout.jsonl` sha1	`02ddcfd613d4784f59bdf8ad20df971b8c9db899` (25 cases; frozen in 9510b6a before its first run)
`golden/amendments.jsonl` sha1	`36a0e6e0d637a48ff43755fd579a1e7b9e80ecce` (one entry, doc-008, commit 45a0392)
corpus: `cat data/docs_versioned/*.md` piped into `shasum`	`c2b025608d03...`; `broker_inventory.csv` `c837aedfa3bd...`
database after 2.3	`16 docs, 16 chunks, 8 superseded, 1 tables`
`b1_check.py`	`PASS`; pair similarity 0.962 to 0.990; old first 1/8
`b1_eval.py` headline	OFF `99/192 (51.6%)`, ON `0/192 (0.0%)`; trap 2/12 to 12/12; control 20/20 both
`pid/make_pid.py`	regenerates byte for byte: `golden_pid.jsonl` sha1 `10fe1aa657131461df6d0b123de96fd2536c06ff`, `ground_truth.json` `1aaa5666aa0360fd237a175ad0188757e8e2c334`, the three drawings concatenated `d6dbc6b6174f...`; 36 tags, 12 questions
versions used	Python 3.12.13, langgraph 1.2.16, openai 3.35.1, psycopg 3.3.6, pgvector (python) 0.5.1, PostgreSQL 16.15, pgvector 0.8.7, Ollama 0.40.6
git	first commit db9af51 is the golden freeze, before any pipeline code; `git log --oneline` in `build/` shows the full order

`recreate.sh` checks the sha1s, the counts and the ablation headline and exits 1 on any mismatch.

5. How the evaluation was kept honest (so you can repeat the method, not just the numbers)

1. The golden set was committed before the pipeline existed (db9af51), and the held-out set before its first run (9510b6a). Git order is the evidence.

2. A scorer bug (doc-008) was fixed through `amendments.jsonl` in its own commit, never by editing the frozen file.

3. Golden questions were never pasted into prompts as examples; a first draft that did was replaced before any run (SOP step 9).

4. Every prompt change was committed before it was measured, and measured on the full set, by failing case id, not only by percentage.

5. Scores are reported as mean and range over repeated runs; the tuned set (88.9%) and the unseen set (81.3%) are reported side by side, and the 7.6-point gap is stated as the measure of over-fitting.

6. Negative results are recorded with their evidence (SOP step 19: qwen3:8b on this CPU).

7. Latency claims come from interleaved OFF/ON timing in one process, never from a single cold run.

6. Hardware and environment limits found on the way

Limit	Evidence	What to do elsewhere
Docling cannot run on Intel macOS	last torch wheel is 2.2.2; Docling's layout model needs transformers for torch 2.4+ and numpy 2 (SOP section 2, runbook section 6)	run Docling on Apple Silicon or Linux and ingest the Markdown; PDFs here use pypdf text only
qwen3:8b cannot grade six passages in usable time	one passage 33-52 s; six passages never returned in 20 min; runner at 578% CPU (SOP step 19)	run `GRADE_MODE=escalate GRADE_MODEL=<bigger>` on a GPU or hosted model; the switch is in place
The Mac sleeps and kills Docker and Ollama connections	kernel `PMRD: System Sleep` in the log matched every outage (runbook 2b)	every long script re-execs under `caffeinate -i`; keep the lid open
A killed client leaves Ollama generating; `chat()` retries re-send it	runner busy 56 min after the client was killed (runbook section 5)	kill by PID, unload with `keep_alive: 0` through the API
A 3B model varies run to run at temperature 0	failing ids change between runs on the tuned set	report ranges; judge changes by failing ids over 3 runs

7. The showcase site

Live on GitHub Pages: https://shreenathacc22.github.io/ragcap/ (static: results, flow, screenshots, and a replay of 64 recorded runs from `site/data/replay.json`). `./publish_site.sh` republishes it; `vercel.json` lets Vercel serve the same `site/` folder after a one-time import in the Vercel dashboard (SOP step 25 has the steps).

8. Related documents

`docs/SOP_ragcap_build.pdf` (every step with problem, change, verification and results), `docs/RUNBOOK_ragcap_operations.pdf` (start/stop, routine tasks, backup, troubleshooting, change control), `docs/build_pdfs.py` (regenerates all three PDFs from this folder; needs `pip install reportlab`), `docs/ragcap_deck.pptx` (the 15-slide deck; `docs/build_deck.js` rebuilds it with pptxgenjs). The analysis of the original design and the pre-build plan are working notes and are not published.